Midterm review - fresh practice, Set 2

Built around your recent written answers: function calls versus pointers, Money return types, abstract classes versus destructors, const access, and bounded array traversal.

Part A: explicit TA-note priorities

A1 Callback repair (p. 2); A2 Money operators (p. 3); A3 Abstract interfaces and destruction (p. 4); A4 Function templates (p. 5). The topics follow your TA notes; these particular exercises are newly written practice, not confirmed exam questions.

Part B: supporting practice - exam inclusion not confirmed

B1 Const pointers and -> (p. 6); B2 Array bounds and sizeof (p. 7); B3 Constructors/static members (p. 8); B4 Lifetime (p. 9); B5 Strings (p. 10); B6 Streams (p. 11). These address observed mistakes and broader revision. Supporting classification does not mean the TA excluded a topic.

How to practise

Use C++17, required standard headers and using namespace std. Assume small arithmetic does not overflow and allocations succeed. No numerical memory addresses are requested. This is a targeted review, not a 100-point mock examination.

Attempt pages 2-11 without the solutions. Write full function definitions where requested. For every output question, state both type and value when asked. Mark invalid code as a compilation error or undefined behavior as appropriate.

Solutions begin on page 12

Each solution uses the same question ID. Compare your explanation as well as your final value. For partial self-checking, distinguish: wrong concept, invalid declaration, missing answer, or incorrect trace.

Evidence for priorities: your C++ Concept Review.md TA notes and the marked mock/interactive submissions in this conversation. Supplementary topics are retained for learning, not presented as new TA predictions.

A1. Callback repair: a function is not its result

```
bool isMultipleOf3(long x) { return x % 3 == 0; }
int countMatches(const long* a, int n,
                 bool (*pred)(long)) {
    int count = 0;
    for (int i = 0; i < n; ++i) {
        if (pred) ++count;
    }
    return count;
}
```

(a) Repair the function with the smallest change. Explain what if(pred) tests and why the original returns n when pred is valid. Assume n >= 0 and a valid range for n > 0.

(b) Declare a long array {-6, 1, 3, 10, 12}. Declare an explicitly typed function pointer named rule, bound to isMultipleOf3. Call the repaired function and state its result.

(c) Write isNegative(long). Reassign rule without redeclaring it, call countMatches again, and give the result. Explain whether rule = isNegative(-6); is valid.

A2. Money: return an object or return its identity

```
struct Money {
    int cents;
    // Write the member operators here.
};
```

(a) Implement + and += inside the struct. Both receive a const-reference right operand. + returns a new Money and accepts a const left operand. += changes the left operand and supports chaining.

(b) Start with Money a{45}, b{12}, c{8}; const Money fee{5}; then Money d = fee + b; and (a += b) += c;. Give final cents for a, b, c, fee and d. Explain why += must return Money& for this contract.

(c) Inside +=, classify this, *this and cents as pointer, object or integer member. Explain why return cents; fails. Is Money const operator+(...) equivalent to a trailing const?

A3. Abstract classes, dispatch and destruction

```
struct Reader {
    virtual int read() const = 0;
    int label() const { return 4; }
    virtual ~Reader() { cout << "R"; }
};
```

(a) Write FixedReader publicly derived from Reader. Override read to return 6, define label to return 9, and write a destructor that prints F. Use override only where valid.

(b) Explain why Reader x; is invalid. Given FixedReader s; Reader& r = s;, state r.read(), r.label(), s.label(). Do these method calls destroy s?

(c) In a separate run: Reader* p = new FixedReader; delete p;. Give the destructor output. If the base destructor is made nonvirtual (and override removed from the derived destructor), is delete p valid? Should you delete twice?

A4. Templates: distinguish deduction from conversion

(a) Write template<typename T> T quarter(T x) returning x / 4. For auto a = quarter(11); auto b = quarter(11.0); double c = quarter(11); auto d = quarter<double>(11); state BOTH type and value.

```
template<typename T>
T smaller(T a, T b) { return a < b ? a : b; }
```

(b) Classify smaller(3, 2.5) and smaller<double>(3, 2.5). Explain the deduction issue; give the result type and value for the valid call.

(c) Write template<typename T> T twice(T x) returning x + x. Give type and value for twice(4) and twice(1.25). Explain why a double destination cannot restore the fraction lost in quarter(11).

B1. Const access, references and the arrow operator

```
struct Box { int value; };
Box a{21}, b{34};
const Box* p = &a;
Box* const q = &a;
```

(a) Independently classify: p = &b; p->value = 7; q = &b; q->value = 7; a.value = 7;. Explain whether a itself is const.

(b) Declare r so neither reseating r nor changing a through r is allowed. Write int readValue(...) taking a pointer with read-only access to a Box. Call it on a and store the returned int.

(c) Rewrite q->value using * and . . Explain how this differs from *q. After a.value = 99;, what does readValue(&a) return? Does reading through a const pointer freeze a?

B2. A zero is data, not an array terminator

```
int values[] = {5, 0, -2, 9};
int* end = values + 4;
int total = 0;
for (int* p = values; *p != 0; ++p) {
    total += *p;
}
```

(a) Give total for this exact code. Repair the loop to process all four elements using p != end. Give the corrected total and explain why zero must not stop it.

(b) Write int sum(const int a[], int n) using n. Handle n = 0 without reading a. Assume n >= 0 and a valid range for n > 0. For this part assume sizeof(int) = 4 and sizeof(const int*) = 8: give sizeof(a), sizeof(a[0]), and their quotient INSIDE sum.

(c) Classify forming end, reading *end, and reading *(end - 1). Explain why an int array does not automatically acquire a null terminator.

B3. Constructor initialization versus assignment

```
struct Pass {
    const int code;
    static int created;
    // Add constructor and getCode.
};
```

(a) Write Pass(int c), initializing code and incrementing created. Add getCode() callable on a const Pass. Define created outside the class, initially zero.

(b) Explain code(c) in the initializer list. Why is Pass(int c) { code = c; ++created; } invalid? Does getCode need to return a Pass object?

(c) Starting from created = 0, construct Pass a(31); const Pass b(42);. Give a.getCode(), b.getCode(), Pass::created. Explain why one shared static member is not a separate counter in each object.

B4. Lifetime: an existing pointer can have a dead target

```
int* makePointer() {
    int x = 23;
    return &x;
}
```

(a) For int* p = makePointer(); cout << *p;, explain exactly which object dies and when. Is the output guaranteed to be 23? Does assigning the pointer to p extend that object's lifetime?

(b) Rewrite the function and caller to safely return 23 by value. Then write int* alias(int& x) returning a pointer to a caller-owned int. Show one safe call and explain its lifetime.

(c) Independently: int* p = new int(23); int* q = p; delete p; p = nullptr;. Describe p and q. May you read *q or delete q? State the minimal local cleanup for q.

B5. Strings: bytes, characters and safe replacement

```
char text[] = "Sun";
string word = "Sun";
```

(a) Give sizeof(text), strlen(text), and the minimum char-array capacity needed for "Moon". Explain why strcpy(text, "Moon") is unsafe.

(b) Write a safe fixed-size char array containing "Moon". Replace word with "Moon" using std::string. Give word.substr(1, 2) and say whether substr changes word.

(c) Classify "Good" + "!", string("Good") + "!", and "Good" + string("!"). Give strcmp("Sun", "Sun"). Explain why a null-terminated C-string loop is different from a bounded int-array loop.

B6. Streams: remaining input and formatting state

```
istringstream in("32\nMai Wong\n");
int id;
string first, name;
in >> id;
getline(in, first);
getline(in, name);
ostringstream out;
out << fixed << setprecision(2) << setfill('_');
out << setw(6) << 7.5 << '|' << 1.0;
```

(a) State id, first, name and out.str(). Explain the first getline result. Which of width, fill, fixed and precision persist after inserting 7.5?

(b) With fresh input "32\nMai Wong\n", write code reading id and then the complete name using one getline. Discard the remaining id line explicitly.

(c) For istringstream nums("4 6 x 8"); int n, total=0; while(nums >> n) total += n;, give total at failure. Write code to clear the failure, consume the single x and conditionally add the remaining integer. Explain why clear alone is insufficient.

A1. Callback repair: a function is not its result

```
// Repair inside the loop:
if (pred(a[i])) ++count;

long data[] = {-6, 1, 3, 10, 12};
bool (*rule)(long) = isMultipleOf3;
int first = countMatches(data, 5, rule); // 3

bool isNegative(long x) { return x < 0; }
// Back in the calling function:
rule = isNegative;
int second = countMatches(data, 5, rule); // 1
```

(a) if(pred) tests whether the function pointer is non-null. It does not call the predicate. Calling pred(a[i]) tests one array element. For n = 0 the loop does not run and count remains zero.

(b) -6, 3 and 12 match. (c) Only -6 is negative. rule = isNegative(-6) is invalid: the call produces bool, not a compatible function pointer. isNegative names the function; isNegative(-6) calls it. Put the isNegative definition outside the calling function.

A2. Money: return an object or return its identity

```
struct Money {
    int cents;
    Money operator+(const Money& rhs) const {
        return Money{cents + rhs.cents};
    }
    Money& operator+=(const Money& rhs) {
        cents += rhs.cents;
        return *this;
    }
};
```

(b) a = 65, b = 12, c = 8, fee = 5, d = 17. += returns a reference to the same left object so the next += modifies that object again. Returning a new copy would not preserve that identity.

(c) In +=, this is Money*, *this denotes the current Money object, and cents is its int member. Returning cents cannot satisfy Money&. In +, cents + rhs.cents is int; Money{...} constructs the required result object. const on the return type does not replace trailing const: trailing const qualifies the member function and allows calling it on fee.

Syntax check: omit Money:: inside the struct. Use Money:: for an out-of-class definition only after declaring the member inside the class. Keep the two const positions separate: rhs is the right operand; trailing const concerns the left object.

A3. Abstract classes, dispatch and destruction

```
struct FixedReader : public Reader {
    int read() const override { return 6; }
    int label() const { return 9; }
    ~FixedReader() override { cout << "F"; }
};
```

(a) label is nonvirtual, so it cannot use override. read must retain the trailing const to match the pure virtual base method. The derived destructor may use override because the base destructor is virtual.

(b) Reader is abstract because read is pure virtual (= 0), not because a destructor exists. Results: 6, 4, 9. The calls do not destroy s. A local s is destroyed when its lifetime ends, normally at scope exit.

(c) Output is FR. One delete invokes the derived destructor, then the base destructor, and releases the allocation. With a nonvirtual base destructor, deleting a FixedReader through Reader* has undefined behavior. Deleting twice never repairs this. Deleting through a FixedReader* would correctly run both destructors even with a nonvirtual base destructor.

A4. Templates: distinguish deduction from conversion

```
template<typename T>
T quarter(T x) { return x / 4; }

template<typename T>
T twice(T x) { return x + x; }
```

(a) a: int 2; b: double 2.75; c: double 2.0; d: double 2.75. For d the explicit type converts 11 to double before division. For c the int division has already produced 2 before the assignment conversion.

(b) smaller(3, 2.5) fails deduction: the arguments propose int and double for one T. smaller<double>(3, 2.5) is valid and returns double 2.5. An explicit T permits argument conversion; deduction does not simply select a common floating-point type here.

(c) twice(4) returns int 8; twice(1.25) returns double 2.5. Integer division discards the fractional part during evaluation. Converting the resulting integer later cannot recover it. On the exam, write the type explicitly rather than using decimal notation alone.

B1. Const access, references and the arrow operator

(a) Valid, invalid, invalid, valid, valid. p can be reseated but cannot modify the Box through p. q always points to a, but may modify a. The original a is not const.

```
const Box* const r = &a;

int readValue(const Box* p) {
    return p->value;
}
// In the caller:
int result = readValue(&a);
// q->value is equivalent to (*q).value
```

(b) r has both const protections. readValue is a free function, so do not add trailing const. Return the int member, not the pointer. Pass &a, not the Box object itself.

(c) *q denotes the entire Box; (*q).value denotes its int member. After a.value = 99, readValue(&a) returns 99. Read-only access through one pointer does not prevent direct modification of the original non-const object.

B2. A zero is data, not an array terminator

```
int total = 0;
for (int* p = values; p != end; ++p) {
    total += *p;
} // total is 12

int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; ++i) total += a[i];
    return total;
}
```

(a) The original loop stops at values[1] and gives 5. The repaired loop includes the zero and negative value and gives 12. With an array containing no zero, the original stopping rule could instead read past the array; it has no general valid bound.

(b) The adjusted parameter type is const int*. With the stated assumptions, sizeof(a) = 8, sizeof(a[0]) = 4, quotient = 2, not the caller's array length 4. Use n. For n = 0 no elements are read.

(c) Forming end is valid. Reading *end has undefined behavior. *(end - 1) is valid and yields 9. Ordinary int arrays have no automatic sentinel element; a zero among the elements is simply a data value.

B3. Constructor initialization versus assignment

```
struct Pass {
    const int code;
    static int created;
    Pass(int c) : code(c) { ++created; }
    int getCode() const { return code; }
};
int Pass::created = 0;
```

(b) code(c) initializes the int member code from parameter c before the constructor body. It is not a call to a function named code. A const member must be initialized; assigning it in the body is not permitted.

getCode returns int because code is an int. A member function does not automatically return the class type merely because it belongs to that class.

(c) The values are 31, 42 and 2. Both constructors increment the same static member. Trailing const allows getCode on b and constrains ordinary modification of the current object through this.

B4. Lifetime: an existing pointer can have a dead target

(a) The local automatic int x dies as makePointer exits. p receives a pointer to that expired object; keeping the pointer does not keep x alive. Reading *p is undefined behavior, not a guaranteed 23. A warning is possible; this need not be a compilation error.

```
int makeValue() { return 23; }
int* alias(int& x) { return &x; }

// In the caller:
int value = makeValue();
int n = 23;
int* observed = alias(n);
cout << *observed; // valid while n is alive
```

(b) alias refers to the caller's int; it does not create a new local int target. That target survives the return. (c) p is null and q dangles. Neither reading *q nor deleting it again is valid. Assign q = nullptr; to clear that alias. Clearing q does not recreate the destroyed allocation.

B5. Strings: bytes, characters and safe replacement

(a) sizeof(text) = 4, strlen(text) = 3. Moon needs five char slots, including the null terminator. strcpy cannot resize the four-element destination; that copy would overflow.

```
char safe[5] = "Moon";
word = "Moon";
string middle = word.substr(1, 2); // "oo"
```

(b) substr returns a separate string and does not change word. Its second argument is a count, not an ending index.

(c) Two string literals cannot be joined with operator+ directly. The two expressions with a std::string operand are valid and produce "Good!". strcmp of equal strings returns 0. A valid C-string supplies a null terminator within accessible storage; an arbitrary int array supplies no such guarantee and may contain zero as ordinary data.

B6. Streams: remaining input and formatting state

(a) id = 32; first is empty; name is "Mai Wong"; out.str() is "__7.50|1.00". Integer extraction leaves a newline, which the first getline consumes as its delimiter. 7.50 has four characters, so width 6 adds two underscores. This insertion resets width; fill, fixed and precision persist.

```
// Fresh input stream:
in >> id;
in.ignore(numeric_limits<streamsize>::max(), '\n');
getline(in, name);

// After the failed numeric loop:
nums.clear();
char bad;
nums >> bad;
if (nums >> n) total += n;
```

(b) Include <limits> for numeric_limits. ignore consumes through the newline; the one getline reads the full name.

(c) total is 10 at failure, then 18 after successful recovery. clear resets flags but does not consume x. Attempting another integer read without removing x would fail again. sizeof(n) measures its type's storage, not whether the next input character is numeric.